

LIBRARY MANAGEMENT SYSTEM

Final Review Report

Submitted by

Priksht Sharma

Registration no.- 19BCIo230

Prepared For

Database Management Systems (CSE2004)

J PROJECT COMPONENT

Submitted to

Dr. Satish C J

Associate Professor

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Table of Contents :-

Page No.

<i>Introduction</i>	<i>3</i>
<i>Project Scope</i>	<i>4</i>
<i>Key Contacts and Stakeholders</i>	<i>4</i>
<i>Project resource requirements</i>	<i>5</i>
<i>ER diagram</i>	<i>9</i>
<i>Tables and constraints</i>	<i>10</i>
<i>Work breakdown</i>	<i>12</i>
<i>Plan to proceed for project</i>	<i>13</i>
<i>Explanation of Er -Diagram</i>	<i>14</i>
<i>Er To Relational Schema</i>	<i>15</i>

<i>Normalization</i>	20
<i>Code for important modules</i>	27
<i>Software snapshots</i>	61

Introduction

This project is a library management system with all the basic as well as some innovative features for managing a library. It consists of a large database of various books available in the library. It also lists various books issued to respective readers. The system keeps track of all the books readily available and also the books that have been issued to various readers for the time period for which the books have been issued. The system also handles books database. If the reader needs a book, he can order the book request for home delivery by just submitting an online form. Readers usually tend to forget the date to return their library books, so this system even calculates fine depending on the expiry date. Thus this innovative library management system provides enhanced library functionality for this modern world.

Project Scope

The scope of Online Library Management System includes:

- 1. Authenticate users at their login.*
- 2. Provide the list of books the users can borrow.*

3. *Facility to reserve books that are available.*
4. *A status page for all users to view books reserved by them.*
5. *Facility to cancel the reservation for a book made earlier.*
6. *A status page for all users to view books borrowed by them, their individual due dates and their individual penalties if any.*
7. *An interface to view and edit the own profile.*

Key Contacts and Stakeholders

<i>Name</i>	<i>Registration Number</i>	<i>Phone number</i>
<i>PRIKSHIT SHARMA</i>	<i>19BCIo230</i>	<i>8968292215</i>

Project Resource requirements

Or

Technologies used

1. *Visual studio*

2. SQL Server
3. C Sharp

Visual Studio Express Edition 2017: -

I have used visual studio because it allows to develop desktop apps in very easy manner. It has some very good features like Intellisense, Auto Complete and many more. More description is as follows :-

Visual Studio Express Edition is one of the most popular platforms designed for passionate programmers. It can be used to develop desktop apps, Metro style programs and hardware drivers.

The GUI looks very familiar for users with plenty of experience in the programming department, but rookies are likely to experience headaches when opening this application.

Fortunately, there is a very helpful online community that's ready to come to your aid with tips and tricks, but you still must have some basic skills in a programming language.

Visual Studio Express Edition comes with features like multi-monitor support (so you can drag and drop documents from one monitor to another), zoom (in editors or diagrams), navigation (through files, code, variables and methods; more efficient than the common

search function), as well as Intellisense (Visual Studio's intuitive feature that simplifies your coding experience).

Visual studio software requirements :-

To develop desktop apps we require to install .NET framework that could be installed from Visual studio installer.

SQL Server Express Edition :-

Microsoft SQL Server 2017 Express is a free, feature-rich editions of SQL Server that is ideal for learning, developing, powering desktop, web & small server applications, and for redistribution by ISVs.

LocalDB (SqlLocalDB)

- LocalDB is a lightweight version of Express that has all its programmability features, yet runs in user mode and has a fast, zero-configuration installation and short list of pre-requisites. Use this if you need a simple way to create and work with databases from code. It can be bundled with Application and Database Development tools like*

Visual Studio and or embedded with an application that needs local databases.

Express (SQLSERVER)

- *Express edition includes the SQL Server database engine only. Best suited to accept remote connections or administer remotely.*

Express with Advanced Services (SQLSERVERADV)

- *This package contains Polybase and Full Text Search features in addition to SQL Database Engine.*

C# :-

C# is pronounced “C-Sharp”.

It is an object-oriented programming language created by Microsoft that runs on the .NET Framework.

C# has roots from the C family, and the language is close to other popular languages like C++ and Java.

The first version was released in year 2002. The latest version, C# 8, was released in September 2019.

C# is used for:

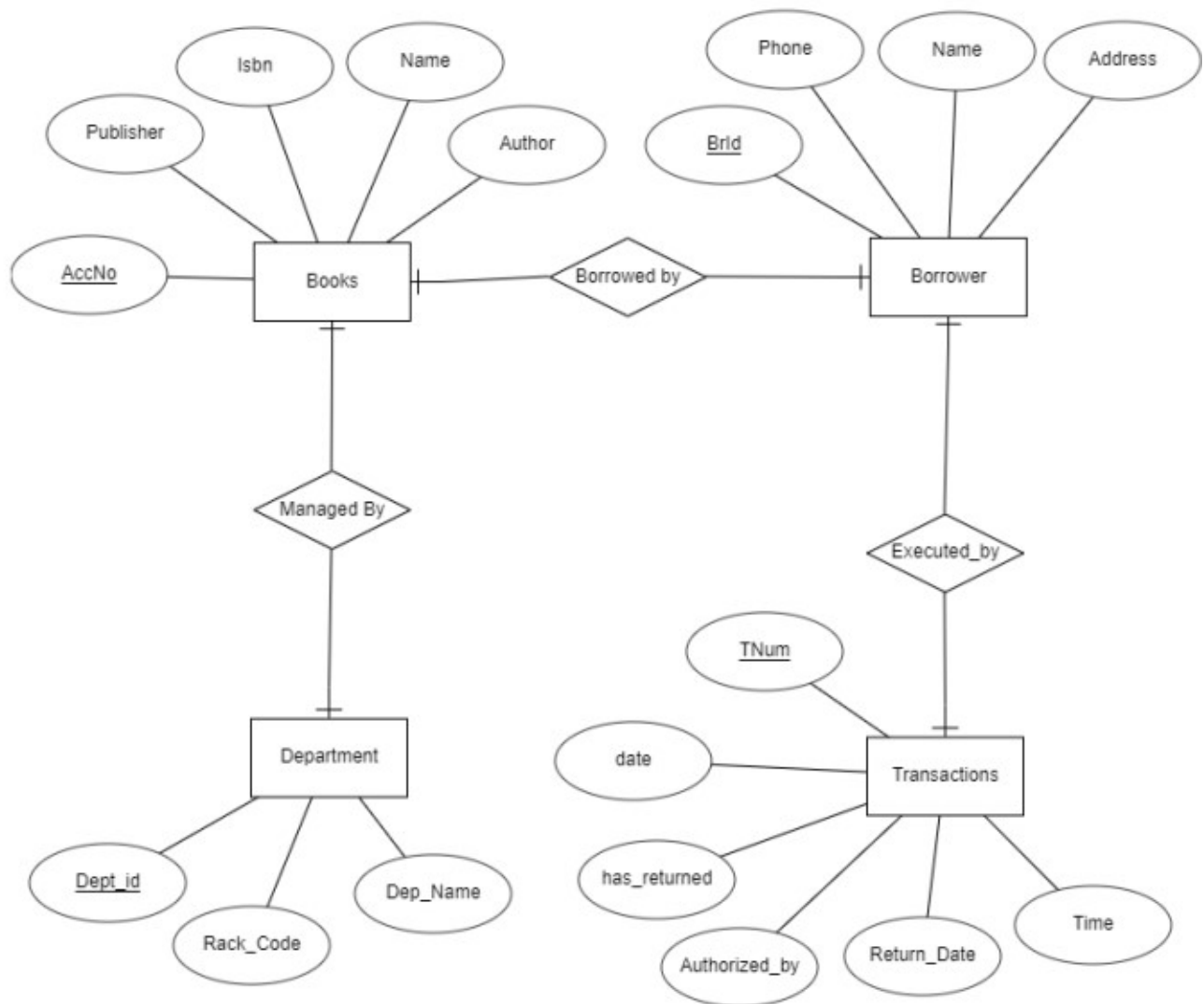
- *Mobile applications*
- *Desktop applications*
- *Web applications*

- *Web services*
- *Web sites*
- *Games*
- *VR*
- *Database applications*
- *And much, much more!*

Why I have used C# ?

1. *It is one of the most popular programming language in the world*
2. *It is easy to learn and simple to use*
3. *It has a huge community support*
4. *C# is an object oriented language which gives a clear structure to programs and allows code to be reused, lowering development costs.*
5. *As C# is close to C, C++ and Java, it makes it easy for programmers to switch to C# or vice versa*

ER DIAGRAM



TABLES AND CONSTRAINTS

1. Books :-

<i>Name</i>	<i>Datatype</i>	<i>Constraints</i>
<i>AccNo</i>	<i>Int</i>	<i>Primary key</i>
<i>Isbn</i>	<i>Varchar(10)</i>	
<i>Name</i>	<i>Varchar(30)</i>	
<i>Author</i>	<i>Varchar(30)</i>	
<i>Publisher</i>	<i>Varchar(30)</i>	
<i>Did</i>	<i>Int</i>	<i>Foreign key</i>
<i>borrower</i>	<i>Int</i>	<i>Foreign key</i>

Foreign key Did references to Dep_id of department
Foreign key borrower to brId of department

2. Transactions :-

<i>Name</i>	<i>Datatype</i>	<i>Constraints</i>
<i>Tnum</i>	<i>Int</i>	<i>Primary key</i>
<i>Date</i>	<i>Date</i>	
<i>Time</i>	<i>Time</i>	
<i>Br_id</i>	<i>Int</i>	<i>Foreign key</i>
<i>Book_num</i>	<i>Int</i>	<i>Foreign key</i>
<i>Authorized_by</i>	<i>Varchar(20)</i>	

<i>Return_date</i>	<i>Date</i>	
<i>Has_returned</i>	<i>Bit</i>	

Foreign key Br_id references to brId of borrower

Foreign key Book_num to AccNo of books

3. Department :-

<i>Name</i>	<i>Datatype</i>	<i>Constraints</i>
<i>Dep_id</i>	<i>Int</i>	
<i>Dep_name</i>	<i>Varchar(10)</i>	
<i>Rack_code</i>	<i>Int</i>	

4. Borrower :-

<i>Name</i>	<i>Datatype</i>	<i>Constraints</i>
<i>brId</i>	<i>Int</i>	<i>Primary key</i>
<i>Name</i>	<i>Varchar(30)</i>	
<i>Address</i>	<i>Varchar(30)</i>	
<i>Phone</i>	<i>Int</i>	
<i>Book1</i>	<i>Int</i>	<i>Foreign key</i>

<i>Book2</i>	<i>int</i>	<i>Foreign key</i>
--------------	------------	--------------------

Foreign key Book1 references AccNo of Books

Foreign key Book2 references AccNo of Books

5. System Table :-

<i>Name</i>	<i>Datatype</i>	<i>Constraints</i>
<i>Property</i>	<i>Varchar(10)</i>	
<i>Value</i>	<i>Varchar(20)</i>	

WORK BREAKDOWN :-

<i>Registration Number</i>	<i>Name</i>	<i>Work Assigned</i>
<i>19BCI0230</i>	<i>PRIKSHIT</i>	<i>WHOLE PROJECT</i>

Plan to proceed for the project :-

Step 1 :-

First create a login form for library management system using Visual studio

Step 2 :-

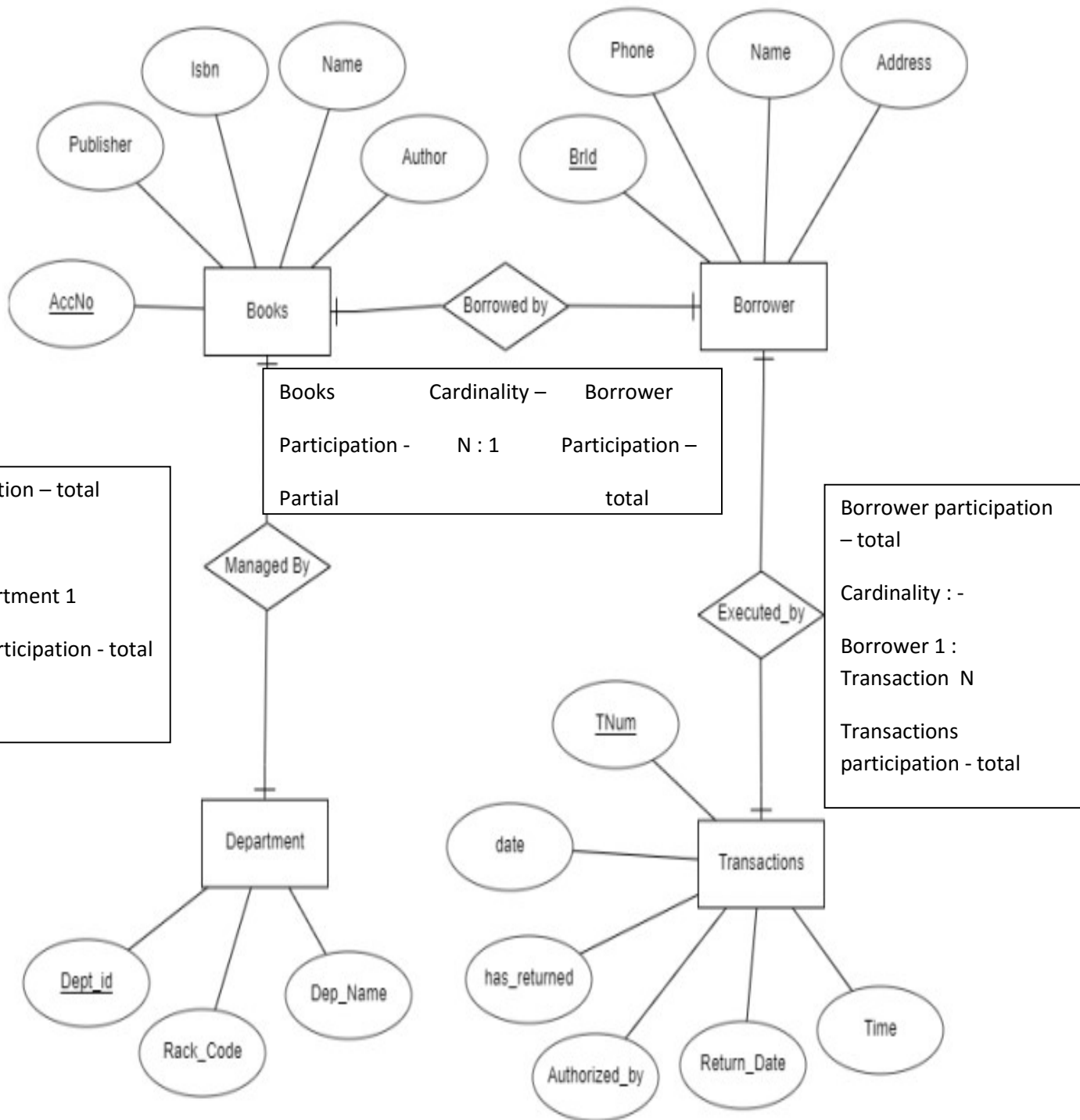
Then add a local database to the form using SQL server express edition

Step 3 :-

Create tables in the local database and

add all the constraints(like primary key,not null and foreign key etc) for each table

Explanation of ER DIAGRAM



ER TO RELATIONAL MAPPING

Following are the steps required to follow to convert an Er diagram to a relational schema

- 1. Mapping of strong entity types*
- 2. Mapping of weak entity types*
- 3. Mapping of 1:1 relationship types*
- 4. Mapping of 1: N relationships types*
- 5. Mapping of M:N relationships types*
- 6. Mapping of multivalued attributes types*
- 7. Mapping of N-ary relationships types*

Step 1 : Mapping of strong entity types

Strong entities are :-

- 1. BOOKS*
- 2. DEPARTMENT*
- 3. TRANSACTIONS*
- 4. BORROWER*

BOOKS TABLE :-

<u>ACCNO</u>	PUBLISHER	ISBN	NAME	AUTHOR
--------------	-----------	------	------	--------

DEPARTMENT TABLE :-

<u>DEPT_ID</u>	RACK_CODE	DEPT_NAME
----------------	-----------	-----------

TRANSACTIONS TABLE :-

<u>TNUM</u>	DATE	HAS_RETURNED	AUTHORIZED_BY	RETURN_DATE	TIME
-------------	------	--------------	---------------	-------------	------

BORROWER TABLE :-

<u>BRID</u>	PHONE	NAME	ADDRESS
-------------	-------	------	---------

SYSTEM TABLE :-

This table is used to store credentials into the database

<u>PROPERTY</u>	VALUE
-----------------	-------

Step 2 : Mapping of weak entity types

There aren't any weak entities

Step 3 : Mapping of 1:1 relationship types

There aren't any such relationship types

Step 4 : Mapping of 1: N relationships types

The following are such relationship types :-

1. *Books : Borrower*
 $N:1$
2. *Books : Department*
 $N:1$
3. *Borrower : Transaction*
 $2.: N$

For such relationship types we include Primary key of entity on side 1 as foreign key attribute on side N relation

So after above mentioned changes the relational schema shall be :-

BOOKS TABLE :-

<u>ACCNO</u>	PUBLISHER	ISBN	NAME	AUTHOR	BR_ID	DEPTID
--------------	-----------	------	------	--------	-------	--------

TRANSACTIONS TABLE :-

<u>TNUM</u>	DATE	HAS_RETURNED	AUTHORIZED_BY	RETURN_DATE	TIME
-------------	------	--------------	---------------	-------------	------

BR_ID

DEPARTMENT TABLE :-

<u><i>DEPT_ID</i></u>	<i>RACK_CODE</i>	<i>DEPT_NAME</i>
-----------------------	------------------	------------------

BORROWER TABLE :-

<u><i>BRID</i></u>	<i>PHONE</i>	<i>NAME</i>	<i>ADDRESS</i>
--------------------	--------------	-------------	----------------

Step 5 : Mapping of M:N relationships types

There aren't any such relationship types

Step 6 : Mapping of multi valued attributes types

There aren't any multi valued attributes

Step 7: Mapping of N-ary relationships types

There aren't any multi valued attributes

Normalization

3. SYSTEM TABLE :-

<i>PROPERTY</i>	<i>VALUE</i>
-----------------	--------------

FD'S:-

PROPERTY $\rightarrow$ {*VALUE*}

1st Normal Form :-

There isn't any composite, multi valued or nested attributes in the relation.

The relation is in 1st normal form

2nd Normal Form :-

Prime attributes - PROPERTY

Non-Prime attributes - VALUE

All Non prime attributes are determined by Primary key

Or

All Non prime attributes are fully dependent on Primary key

The relation is in 2nd normal form

3rd Normal Form :-

No 3rd attribute is there for transitive dependency to exist

The relation is in 3rd normal form

4. BOOKS TABLE :-

<u>ACCNO</u>	PUBLISHER	ISBN	NAME	AUTHOR	BR_ID	DEPTID
--------------	-----------	------	------	--------	-------	--------

FD'S :-

$ACCNO \rightarrow \{PUBLISHER, ISBN, NAME, AUTHOR, BR_ID, DEP_ID\}$

Inferenced FD'S -

$ACCNO \rightarrow \{PUBLISHER\}, ACCNO \rightarrow \{ISBN\}, ACCNO \rightarrow \{NAME\},$

$ACCNO \rightarrow \{AUTHOR\}, ACCNO \rightarrow \{BR_ID\}, ACCNO \rightarrow \{DEP_ID\}$

1st Normal Form :-

There isn't any composite, multi valued or nested attributes in the relation.

The relation is in 1st normal form

2nd Normal Form :-

Prime attributes - ACCNO

Non-Prime attributes - PUBLISHER, ISBN, NAME, AUTHOR, BR_ID and DEP_ID

All Non prime attributes are determined by Primary key and not by any subset of primary key

Or

All Non prime attributes are fully dependent on Primary key

The relation is in 2nd normal form

3rd Normal Form :-

No non-prime attribute is transitively dependent on primary key

The relation is in 3rd normal form

5. TRANSACTIONS TABLE :-

<u>TNUM</u>	DATE	HAS_RETURNED	AUTHORIZED_BY	RETURN_DATE	TIME

BR_ID

FD'S :-

$TNUM \rightarrow \{DATE, HAS_RETURNED, AUTHORIZED_BY, RETURN_DATE, TIME, BR_ID\}$

Inferenced FD'S -

$TNUM \rightarrow \{DATE\}, TNUM \rightarrow \{HAS_RETURNED\}, TNUM \rightarrow \{RETURN_DATE\}$

$TNUM \rightarrow \{AUTHORIZED_BY\}, TNUM \rightarrow \{BR_ID\}$

1st Normal Form :-

There isn't any composite, multi valued or nested attributes in the relation.

The relation is in 1st normal form

2nd Normal Form :-

Prime attributes - TNUM

Non-Prime attributes - DATE, HAS_RETURNED, AUTHORIZED_BY, RETURN_DATE and TIME

All Non-prime attributes are determined by Primary key

Or

All Non-prime attributes are fully dependent on Primary key

The relation is in 2nd normal form

3rd Normal Form :-

No non-prime attribute is transitively dependent on primary key

The relation is in 3rd normal form

4. *DEPARTMENT TABLE :-*

<u><i>DEPT_ID</i></u>	<i>RACK_CODE</i>	<i>DEPT_NAME</i>
-----------------------	------------------	------------------

FD'S :-

DEPT_ID $\rightarrow$ {*RACK_CODE*, *DEPT_ID*}

Inferenced FD'S -

DEPT_ID $\rightarrow$ {*RACK_CODE*}, *DEPT_ID* $\rightarrow$ {*DEPT_ID*}

1st Normal Form :-

There isn't any composite, multi valued or nested attributes in the relation.

The relation is in 1st normal form

2nd Normal Form :-

Prime attributes - DEPT_ID

Non-Prime attributes - RACK_CODE, DEPT_NAME

All Non-prime attributes are determined by Primary key

Or

All Non-prime attributes are fully dependent on Primary key

The relation is in 2nd normal form

3rd Normal Form :-

No non-prime attribute is transitively dependent on primary key

The relation is in 3rd normal form

5. BORROWER TABLE :-

<u><i>BRID</i></u>	<i>PHONE</i>	<i>NAME</i>	<i>ADDRESS</i>
--------------------	--------------	-------------	----------------

FD'S :-

BRID $\rightarrow$ {PHONE, NAME, ADDRESS}

Inferenced FD'S -

BRID $\rightarrow$ {PHONE}, BRID $\rightarrow$ {NAME}, BRID $\rightarrow$ {ADDRESS}

1st Normal Form :-

There isn't any composite, multi valued or nested attributes in the relation.

The relation is in 1st normal form

2nd Normal Form :-

Prime attributes - BRID

Non-Prime attributes - PHONE, NAME, ADDRESS

All Non-prime attributes are determined by Primary key

Or

All Non-prime attributes are fully dependent on Primary key

The relation is in 2nd normal form

3rd Normal Form :-

No non-prime attribute is transitively dependent on primary key

The relation is in 3rd normal form

LOGIN FORM CODE

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;

namespace Library_Management_System
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=\\|DataDirectory|\\DBMS1.mdf\\";Integrated
Security=True");
            SqlCommand cmd;
            SqlDataReader dr;

            private String getUsername()
            {
                con.Open();
                String syntax = "select value from SYSTEMTABLE where property='USERNAME'";
                cmd = new SqlCommand(syntax, con);
                dr = cmd.ExecuteReader();
                dr.Read();
                String temp= dr[0].ToString();
                con.Close();
                return temp;
            }

            private String getPassword()
            {
                con.Open();
                String syntax = "select value from SYSTEMTABLE where property='PASSWORD'";
                cmd = new SqlCommand(syntax, con);
                dr = cmd.ExecuteReader();
                dr.Read();
                String temp = dr[0].ToString();
                con.Close();
                return temp;
            }

            private void button1_Click(object sender, EventArgs e)

```

```

{
    String Uname = getUsername(), Upass = getPassword(), name, pass;
    name = textBox2.Text;
    pass = textBox3.Text;

    if (Uname.Equals(name) && Upass.Equals(pass))
    {
        label15.Hide();
        APPBODY obj = new APPBODY();
        this.Hide();
        obj.Show();
    }
    else
    {
        label15.Show();
    }
}
}
}

```

MAIN APP BODY CODE

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Runtime.InteropServices;

namespace Library_Management_System
{
    public partial class APPBODY : Form
    {
        public const int WM_NCLBUTTONDOWN = 0xA1;
        public const int HT_CAPTION = 0x2;

        [DllImportAttribute("user32.dll")]
        public static extern int SendMessage(IntPtr hWnd,
            int Msg, int wParam, int lParam);
        [DllImportAttribute("user32.dll")]
        public static extern bool ReleaseCapture();

        public APPBODY()
        {
            InitializeComponent();

            isSlidingPanelExpanded = true;
            expandSlidingGUI();
        }
    }
}

```

```

}

private void APPBODY_Load(object sender, EventArgs e)
{
    this.Opacity = 0.1;
    LOGIN_TIMER.Start();
}
private void LOGIN_TIMER_Tick(object sender, EventArgs e)
{
    if (this.Opacity <= 1.0)
    {
        this.Opacity += 0.025;
    }
    else
    {
        LOGIN_TIMER.Stop();
    }
}

private void CLOSEBUTTON_Click(object sender, EventArgs e)
{
    Environment.Exit(0);
}

private void MINIMIZEBUTTON_Click(object sender, EventArgs e)
{
    this.WindowState = FormWindowState.Minimized;
}

private void LOGOUTBUTTON_Click(object sender, EventArgs e)
{
    Form1 obj = new Form1();
    this.Hide();
    obj.Show();
}

private void TITLEPANEL_MouseDown(object sender, MouseEventArgs e)
{
    if (e.Button == MouseButtons.Left)
    {
        ReleaseCapture();
        SendMessage(Handle, WM_NCLBUTTONDOWN, HT_CAPTION, 0);
    }
}

private void panel1_Paint(object sender, PaintEventArgs e)
{
}

private void expandSlidingGUI()
{
    BOOKSTAB.Text = "BOOKS";
    BORROWERTAB.Text = "BORROWERS";
    TRANSACTIONSTAB.Text = "TRANSACTIONS";
    SETTINGSTAB.Text = "SETTINGS";
    ABOUTTAB.Text = "ABOUT";
}

```

```

        SLIDINGTOGGLE.Image = Properties.Resources.arleft;
        BOOKSTAB.Image = null;
        BORROWERTAB.Image = null;
        TRANSACTIONSTAB.Image = null;
        SETTINGSTAB.Image = null;
        ABOUTTAB.Image = null;
    }

    private void retractSlidingGUI()
    {
        BOOKSTAB.Text = "";
        BORROWERTAB.Text = "";
        TRANSACTIONSTAB.Text = "";
        SETTINGSTAB.Text = "";
        ABOUTTAB.Text = "";

        SLIDINGTOGGLE.Image = Properties.Resources.rightAr;
        BOOKSTAB.Image = Properties.Resources.book2;
        BORROWERTAB.Image = Properties.Resources.Users;
        TRANSACTIONSTAB.Image = Properties.Resources.transtn;
        SETTINGSTAB.Image = Properties.Resources.setting;
        ABOUTTAB.Image = Properties.Resources.informtn;
    }

    // SLIDING PANEL CODE STARTS HERE
    bool isSlidingPanelExpanded;
    const int MaxSliderWidth = 275;
    const int MinSliderWidth = 85;

    private void SLIDINGTOGGLE_Click(object sender, EventArgs e)
    {
        if (isSlidingPanelExpanded)
        {
            // retract panel
            retractSlidingGUI();
        }
        SLIDINGTIMER.Start();
    }

    private void SLIDINGTIMER_Tick(object sender, EventArgs e)
    {
        if (isSlidingPanelExpanded)
        {
            //retract panel
            SLIDINGPANEL.Width -= 75;
            if (SLIDINGPANEL.Width <= MinSliderWidth)
            {
                // stop retracting
                isSlidingPanelExpanded = false;
                SLIDINGTIMER.Stop();

                this.Refresh();
            }
        }
    }

```

```

    }
    else
    {
        // expand the panel
        SLIDINGPANEL.Width += 75;
        if (SLIDINGPANEL.Width >= MaxSliderWidth)
        {
            // stop expanding
            isSlidingPanelExpanded = true;
            SLIDINGTIMER.Stop();
            expandSlidingGUI();
            this.Refresh();
        }
    }
}

private void label2_Click(object sender, EventArgs e)
{
}

private void BOOKSTAB_Click_1(object sender, EventArgs e)
{
    if (!panel1.Controls.Contains(BOOKS_USERCONTROL.Instance))
    {
        panel1.Controls.Add(BOOKS_USERCONTROL.Instance);
        BOOKS_USERCONTROL.Instance.Dock = DockStyle.Fill;
        BOOKS_USERCONTROL.Instance.BringToFront();
    }
    else
    {
        BOOKS_USERCONTROL.Instance.BringToFront();
    }
}

private void BORROWERTAB_Click_1(object sender, EventArgs e)
{
    if (!panel1.Controls.Contains(BORROWERS_UserControl.Instance))
    {
        panel1.Controls.Add(BORROWERS_UserControl.Instance);
        BORROWERS_UserControl.Instance.Dock = DockStyle.Fill;
        BORROWERS_UserControl.Instance.BringToFront();
    }
    else
    {
        BORROWERS_UserControl.Instance.BringToFront();
    }
}

private void TRANSACTIONSTAB_Click_1(object sender, EventArgs e)
{
    if (!panel1.Controls.Contains(TRANSACTION_USERCONTROL.Instance))
    {
        panel1.Controls.Add(TRANSACTION_USERCONTROL.Instance);
    }
}

```

```

        TRANSACTION_USERCONTROL.Instance.Dock = DockStyle.Fill;
        TRANSACTION_USERCONTROL.Instance.BringToFront();
    }
    else
    {
        TRANSACTION_USERCONTROL.Instance.BringToFront();
    }
}

private void SETTINGSTAB_Click_1(object sender, EventArgs e)
{
    if (!panel1.Controls.Contains(DEPARTMENT_USER_CONTROL.Instance))
    {
        panel1.Controls.Add(DEPARTMENT_USER_CONTROL.Instance);
        DEPARTMENT_USER_CONTROL.Instance.Dock = DockStyle.Fill;
        DEPARTMENT_USER_CONTROL.Instance.BringToFront();
    }
    else
    {
        DEPARTMENT_USER_CONTROL.Instance.BringToFront();
    }
}

private void ABOUTTAB_Click_1(object sender, EventArgs e)
{
    if (!panel1.Controls.Contains(ABOUT_USERCONTROLS.Instance))
    {
        panel1.Controls.Add(ABOUT_USERCONTROLS.Instance);
        ABOUT_USERCONTROLS.Instance.Dock = DockStyle.Fill;
        ABOUT_USERCONTROLS.Instance.BringToFront();
    }
    else
    {
        ABOUT_USERCONTROLS.Instance.BringToFront();
    }
}

private void button1_Click(object sender, EventArgs e)
{
}

private void panel1_Paint_1(object sender, PaintEventArgs e)
{
}
}
}

```


The code for various user controls I have used

BOOKS USER CONTROL :-

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;

namespace Library_Management_System
{
    public partial class BOOKS_USERCONTROL : UserControl
    {
        private static BOOKS_USERCONTROL _instance;

        public static BOOKS_USERCONTROL Instance
        {
            get
            {
                if(_instance==null)
                {
                    _instance = new BOOKS_USERCONTROL();
                }
                return _instance;
            }
        }

        public BOOKS_USERCONTROL()
        {
            InitializeComponent();

            SqlConnection con = new SqlConnection(@"Data Source =
(LocalDB)\MSSQLLocalDB;AttachDbFilename=|DataDirectory|\DBMS1.mdf;Integrated Security =
True");

            // GENERAL VIEW FUNCTIONALITY

            public void refresh_DataGridView()    // SELECTING ALL THE DATA AND DISPLAYING IT
ON THE SCREEN
            {
                try
                {
                    SqlCommand cmd = new SqlCommand("SHOWALLBOOKSDATA", con);
                    cmd.CommandType = CommandType.StoredProcedure;

                    SqlDataAdapter DA = new SqlDataAdapter(cmd);
                    DataSet DS = new DataSet();
```

```

        DA.Fill(DS);
        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" + exp);
        }
        con.Close();

        DATAGRIDVIEW1.DataSource = DS.Tables[0];           // Below is code for
front end view of the books menu
        this.DATAGRIDVIEW1.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[3].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[4].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[5].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[6].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[7].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

    }
    catch (Exception exp)
    {
        MessageBox.Show(" " + exp);
    }
}

private void button2_Click(object sender, EventArgs e)           // Update button code
{
    SqlCommand cmd = new SqlCommand("UPDATE_BOOKS_SP", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@ACCNO", ACCNOTEXTB.Text);
    cmd.Parameters.AddWithValue("@ISBN", ISBNTEXTB.Text);
    cmd.Parameters.AddWithValue("@NAME", NAMETEXTB.Text);
    cmd.Parameters.AddWithValue("@AUTHOR", AUTHORTEXTB.Text);
    cmd.Parameters.AddWithValue("@PUBLISHER", PUBLISHERTEXTB.Text);
    cmd.Parameters.AddWithValue("@DID", DEPARTMENTTEXTB.Text);

    con.Open();
    try
    {
        cmd.ExecuteNonQuery();
    }
    catch (Exception exp)           // To handle exceptions of SQL
    {

```

QUERIES

```

        MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
    }
    con.Close();

    refresh_DataGridView();
}

code
private void button1_Click(object sender, EventArgs e)           // Add new button
{
    SqlCommand cmd = new SqlCommand("BOOKSADD_SP", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@ACCNO", ACCNOTEXTB.Text);
    cmd.Parameters.AddWithValue("@ISBN", ISBNTEXTB.Text);
    cmd.Parameters.AddWithValue("@NAME", NAMETEXTB.Text);
    cmd.Parameters.AddWithValue("@AUTHOR", AUTHORETEXTB.Text);
    cmd.Parameters.AddWithValue("@PUBLISHER", PUBLISHERTEXTB.Text);
    cmd.Parameters.AddWithValue("@DID", DEPARTMENTTEXTB.Text);

    con.Open();
    try
    {
        cmd.ExecuteNonQuery();
    }
    catch(Exception exp)                                         // To handle exceptions of SQL
    {
        MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
    }
    con.Close();

    refresh_DataGridView();
}

QUERIES
private void BOOKS_USERCONTROL_Load(object sender, EventArgs e) // Code
executed at the time books user control will be loaded
{
    refresh_DataGridView();
}

e)
private void DATAGRIDVIEW1_CellContentClick(object sender, DataGridViewCellEventArgs
{
}

private void DELETEBUTTON_Click(object sender, EventArgs e)
{
    try
    {
        SqlCommand cmd = new SqlCommand("BOOKSDELETE_SP", con);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@ACCNO", ACCNOTEXTB.Text);

```

```

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
// To handle exceptions of SQL QUERIES
        }
        con.Close();

        refresh_DataGridView();

    }
    catch (Exception exp)                // To handle exceptions of C SHARP
    {
        MessageBox.Show(" " + exp);
    }
}

private void SEARCHBOOKBUTTON_Click(object sender, EventArgs e)
{
    try                                // SEARCHING FOR A SPECIFIC TUPLE OR RECORD IN THE TABLE
    {
        SqlCommand cmd = new SqlCommand("SEARCHBOOKS_SP", con);
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@ACCNO", BOOKIDTEXTB.Text);

        SqlDataAdapter DA = new SqlDataAdapter(cmd);
        DataSet DS = new DataSet();
        DA.Fill(DS);

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" + exp);
        }
        con.Close();

        DATAGRIDVIEW1.DataSource = DS.Tables[0];
        this.DATAGRIDVIEW1.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[3].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
    }
}

```

```

        this.DATAGRIDVIEW1.Columns[4].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[5].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[6].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.DATAGRIDVIEW1.Columns[7].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

    }
    catch (Exception exp)
    {
        MessageBox.Show(" " + exp);
    }
}

private void CLEARBUTTON_Click(object sender, EventArgs e)
{
    BOOKIDTEXTB.Text = " ";
    ACCNOTEXTB.Text = " ";
    PUBLISHERTEXTB.Text = " ";
    ISBNTEXTB.Text = " ";
    AUTHORTEXTB.Text = " ";
    DEPARTMENTTEXTB.Text = " ";
    NAMETEXTB.Text = " ";
}

private void ACCNOTEXTB_TextChanged(object sender, EventArgs e)
{
}

private void PUBLISHERLABEL_Click(object sender, EventArgs e)
{
}

private void AUTHORLABEL_Click(object sender, EventArgs e)
{
}

private void ACCNOLABEL_Click(object sender, EventArgs e)
{
}

private void BOOKIDLABEL_Click(object sender, EventArgs e)
{
}

private void button1_Click_1(object sender, EventArgs e) // Refresh button
code will be executed
{
    refresh_DataGridView();
}

```

```

    }
}

```

BORROWERS USER CONTROL CODE :-

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;

namespace Library_Management_System
{
    public partial class BORROWERS_UserControl : UserControl
    {
        private static BORROWERS_UserControl _instance;

        public static BORROWERS_UserControl Instance
        {
            get
            {
                if (_instance == null)
                {
                    _instance = new BORROWERS_UserControl();
                }
                return _instance;
            }
        }

        public BORROWERS_UserControl()
        {
            InitializeComponent();

            SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=\\DataDirectory\\DBMS1.mdf\\";Integrated
Security=True");
            SqlCommand cmd;

            public void refresh_DataGridView()    // SELECTING ALL THE DATA AND DISPLAYING IT
ON THE SCREEN
            {
                try
                {
                    SqlCommand cmd = new SqlCommand("SHOW_ALL_BORROWERS_DATA", con);    //
The STORED PROCEDURE- SHOW ALL BORROWER'S DATA

```

//

IS YET TO BE CODED

```

        cmd.CommandType = CommandType.StoredProcedure;

        SqlDataAdapter DA = new SqlDataAdapter(cmd);
        DataSet DS = new DataSet();
        DA.Fill(DS);
        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" + exp);
        }
        con.Close();

        BORROWERTABLEBRTB.DataSource = DS.Tables[0];           // Below is code for
front end view of the books menu
        this.BORROWERTABLEBRTB.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.BORROWERTABLEBRTB.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.BORROWERTABLEBRTB.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.BORROWERTABLEBRTB.Columns[3].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

    }
    catch (Exception exp)
    {
        MessageBox.Show(" " + exp);
    }
}

private void BORROWERS_UserControl_Load(object sender, EventArgs e)
{
    refresh_DataGridView();
}

private void label1_Click(object sender, EventArgs e)
{
}

private void DELETEBTNBRBTB_Click(object sender, EventArgs e)
{
    try
    {
        SqlCommand cmd = new SqlCommand("DELETE_BORROWERS_SP", con);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@BRID", BRIDTEXTB2BRTB.Text);

```

```

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
// To handle exceptions of SQL QUERIES
        }
        con.Close();

        refresh_DataGridView();

    }
    catch (Exception exp)                // To handle exceptions of C SHARP
    {
        MessageBox.Show(" " + exp);
    }
}

private void CLEARBTNBRTB_Click(object sender, EventArgs e)
{
    BRIDTEXTBBRTB.Text = " ";
    BRIDTEXTB2BRTB.Text = " ";
    NAMETEXTBBRTB.Text = " ";
    ADDRESSTEXTBBRTB.Text = " ";
    PHONETEXTBBRTB.Text = " ";
}

private void ADDBORROWERBRTB_Click(object sender, EventArgs e)
{
    SqlCommand cmd = new SqlCommand("ADD_BORROWERS_SP", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@BRID", BRIDTEXTB2BRTB.Text);
    cmd.Parameters.AddWithValue("@NAME", NAMETEXTBBRTB.Text);
    cmd.Parameters.AddWithValue("@ADDRESS", ADDRESSTEXTBBRTB.Text);
    cmd.Parameters.AddWithValue("@PHONE", PHONETEXTBBRTB.Text);

    con.Open();
    try
    {
        cmd.ExecuteNonQuery();
    }
    catch (Exception exp)                // To handle exceptions of SQL
    {
        MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
    }
    con.Close();

    refresh_DataGridView();
}

```

QUERIES


```

}

private void REFRESHBTNBRTB_Click(object sender, EventArgs e)
{
    refresh_DataGridView();
}

private void SEARCHBORWRBRTB_Click(object sender, EventArgs e)
{
    try // SEARCHING FOR A SPECIFIC TUPLE OR RECORD IN THE TABLE
    {
        SqlCommand cmd = new SqlCommand("SEARCH_BORROWERS", con);
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@BRID", BRIDTEXTBRTB.Text);

        SqlDataAdapter DA = new SqlDataAdapter(cmd);
        DataSet DS = new DataSet();
        DA.Fill(DS);

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" + exp);
        }
        con.Close();

        BORROWERTABLEBRTB.DataSource = DS.Tables[0];
        this.BORROWERTABLEBRTB.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.BORROWERTABLEBRTB.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.BORROWERTABLEBRTB.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
        this.BORROWERTABLEBRTB.Columns[3].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

    }
    catch (Exception exp)
    {
        MessageBox.Show(" " + exp);
    }
}

private void UPDATEBTNBRTB_Click(object sender, EventArgs e)
{
    SqlCommand cmd = new SqlCommand("UPDATE_BORROWERS_SP", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@BRID", BRIDTEXTB2BRTB.Text);
    cmd.Parameters.AddWithValue("@NAME", NAMETEXTBBRTB.Text);
    cmd.Parameters.AddWithValue("@ADDRESS", ADDRESSTEXTBBRTB.Text);
    cmd.Parameters.AddWithValue("@PHONE", PHONETEXTBBRTB.Text);

```

```

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)                                // To handle exceptions of SQL
        {
            MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
        }
        con.Close();

        refresh_DataGridView();
    }
}

```

QUERIES

TRANSACTIONS USER CONTROL CODE :-

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;
namespace Library_Management_System
{
    public partial class TRANSACTION_USERCONTROL : UserControl

```

```

{
    private static TRANSACTION_USERCONTROL _instance;

    public static TRANSACTION_USERCONTROL Instance
    {
        get
        {
            if (_instance == null)
            {
                _instance = new TRANSACTION_USERCONTROL();
            }
            return _instance;
        }
    }

    public TRANSACTION_USERCONTROL()
    {
        InitializeComponent();
    }

    public String Book1, Book2, Borrower;

    SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=\\\"|DataDirectory|\\DBMS1.mdf\\\";I
ntegrated Security=True");

    SqlCommand cmd;

    private void SEARCHBOOK2BTN_Click(object sender, EventArgs e)

```

```

{
    // To get borrowed by
    con.Open();

    String syntax = "SELECT BORROWER FROM BOOKS WHERE ACCNO='" +
    ACCNOTEXTBTRNCTB.Text + "'"; // CODE IS SIMILAR TO FORM1 CODE

    cmd = new SqlCommand(syntax, con);

    dr = cmd.ExecuteReader();

    dr.Read();

    BORROWEDBYLABELFTRNCTB.Text = Borrower = dr[0].ToString();

    con.Close();

}

```

```

private void CLEARALLBTNTRNCTB_Click(object sender, EventArgs e)

```

```

{
    BRIDTEXTBTRNCTB.Text = "";
    ACCNOTEXTBTRNCTB.Text = "";
    BOOK1LABELFTRNCTB.Text = "";
    BOOK2LABELFTRNCTB.Text = "";
    BORROWEDBYLABELFTRNCTB.Text = "";
}

```

```

private void ISSUEBOOKBTN_Click(object sender, EventArgs e)

```

```

{
    // Check whether the book has been borrowed by someone
    SEARCHBOOK2BTN.PerformClick();
}

```

```

        if(Borrower!="")           // there's a difference in null and " " -
null string
    {
        // Someone has borrowed the book

        MessageBox.Show("Book is already borrowed by some other borrower
with borrower id : \n" + Borrower);

        return;
    }

    // See if the borrower has already taken 2 books
    SEARCHBOOK2BTN.PerformClick();
    if((Book1 != "") && (Book2 != ""))
    {
        // Borrower has already taken maximum number of books that is he
has already taken 2 books

        MessageBox.Show("Sorry for nconvenience, Borrower has already
borrowed maximum number of books that is he/she has already borrowed 2 books");

        return;
    }

    // all is well, now we can proceed for the ISSUE process

    // first set the value of the book1 or book2 as the accno of the
book being issued

    try
    {

```

```

        if(Book1=="")
        {
            cmd = new SqlCommand("TRANSACT_UPDATE_BOOK1_SP", con);
        }
        else
        {
            cmd = new SqlCommand("TRANSACT_UPDATE_BOOK2_SP", con);
        }
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@ACCNO", ACCNOTEXTBTRNCTB.Text);
        cmd.Parameters.AddWithValue("@BRID", BRIDTEXTBTRNCTB.Text);

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch(Exception exp)
        {
            MessageBox.Show("
: \n"+exp);
            <<<<  INVALID SQL OPERATION  >>>>
        }
        con.Close();
    }
    catch(Exception exp)

```

```

{
    MessageBox.Show(“” + exp);
}

// NOW WE SHALL UPDATE THE BORROWER’S ID IN BOOKS TABLE

cmd = new SqlCommand(“TRANSACT_UPDATE_BORROWER_SP”, con);
cmd.CommandType = CommandType.StoredProcedure;

cmd.Parameters.AddWithValue(“@ACCNO”, ACCNOTEXTBTRNCTB.Text);
cmd.Parameters.AddWithValue(“@BRID”, BRIDTEXTBTRNCTB.Text);

con.Open();
try
{
    cmd.ExecuteNonQuery();
}
catch (Exception exp)
{
    MessageBox.Show(“<<<<  INVALID SQL OPERATION  >>>> :
\n”+exp);
}
con.Close();

```



```

private void RETURNBOOKBTN_Click(object sender, EventArgs e)
{

    // Check whether the book has been borrowed by someone
    SEARCHBOOK2BTN.PerformClick();
    SEARCHBORWERBTN.PerformClick();

    if ((ACCNOTEXTBTRNCTB.Text != Book1) && (ACCNOTEXTBTRNCTB.Text
!=Book2))    // there's a difference in null and " " - null string
    {
        // Someone has borrowed the book

        MessageBox.Show("THE INPUTTED BORROWER HAS NOT BORROWED THE BOOK
INPUTED");

        return;
    }


    // all is well, now we can proceed for the RETURN process


    // Update the borrower table

    // first set the value of the book1 or book2 as the null of the
book being returned


    try
    {

        if (Book1 == ACCNOTEXTBTRNCTB.Text)

```

```

{
    // accno must be updated in book1 slot
    cmd = new SqlCommand("TRANSACT_UPDATE_BOOK1_SP", con);
}
else
{
    // accno must be updated in book1 slot
    cmd = new SqlCommand("TRANSACT_UPDATE_BOOK2_SP", con);
}
cmd.CommandType = CommandType.StoredProcedure;

cmd.Parameters.AddWithValue("@ACCNO", (object)DBNull.Value);
cmd.Parameters.AddWithValue("@BRID", BRIDTEXTBTRNCTB.Text);

con.Open();
try
{
    cmd.ExecuteNonQuery();
}
catch (Exception exp)
{
    MessageBox.Show("
: \n" + exp);
}
con.Close();
}

```

```

catch (Exception exp)
{
    MessageBox.Show(“” + exp);
}

// NOW WE SHALL UPDATE THE BORROWER’S ID IN BOOKS TABLE

cmd = new SqlCommand(“TRANSACT_UPDATE_BORROWER_SP”, con);
cmd.CommandType = CommandType.StoredProcedure;

cmd.Parameters.AddWithValue(“@ACCNO”, ACCNOTEXTBTRNCTB.Text);
cmd.Parameters.AddWithValue(“@BRID”, (object)DBNull.Value);

con.Open();
try
{
    cmd.ExecuteNonQuery();
}
catch (Exception exp)
{
    MessageBox.Show(“<<<< INVALID SQL OPERATION >>>> :
\n” + exp);
}
con.Close();

```

```

        cmd = new SqlCommand("TRANSACTIONS_DELETE_SP", con);
// USE OF TRANSACTION_DELETE_SP

        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@BKID", ACCNOTEXTBTRNCTB.Text);
        cmd.Parameters.AddWithValue("@BRID", (object)DBNull.Value);

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("
<<<<  INVALID SQL OPERATION  >>>> :
\n" + exp);
        }
        con.Close();

// Below lines of code were not in actual code

// SEARCHBOOK2BTN.PerformClick();
// SEARCHBORWERBTN.PerformClick();
// MessageBox.Show("Successfully returned");

```

```

    }

    public void refresh_DataGridView()      // SELECTING ALL THE DATA AND
DISPLAYING IT ON THE SCREEN
    {
        try
        {
            SqlCommand cmd = new
SqlCommand("SHOWTRANSACTION_TRIGGER_DETAILS", con);      // The STORED
PROCEDURE- SHOW ALL BORROWER'S DATA

//      IS YET TO BE CODED

            cmd.CommandType = CommandType.StoredProcedure;

            SqlDataAdapter DA = new SqlDataAdapter(cmd);
            DataSet DS = new DataSet();
            DA.Fill(DS);
            con.Open();
            try
            {
                cmd.ExecuteNonQuery();
            }
            catch (Exception exp)
            {
                MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" +
exp);
            }
        }
    }

```

```

        con.Close();

        TRNCLOGTABLE.DataSource = DS.Tables[0];           // Below is
code for front end view of the books menu

        this.TRNCLOGTABLE.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

        this.TRNCLOGTABLE.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

    }

    catch (Exception exp)
    {
        MessageBox.Show("“" + exp);
    }

}

private void SHOWTRNCBTN_Click(object sender, EventArgs e)
{
    refresh_DataGridView();
}

private void REFRESHBTNBRTB_Click(object sender, EventArgs e)

```

```

{

}

SqlDataReader dr;

private void SEARCHBORWERBTN_Click(object sender, EventArgs e)
{
    // Top get Book 1
    con.Open();

    String syntax = "SELECT BOOK1 FROM BORROWER WHERE BRID='" +
BRIDTEXTBTRNCTB.Text+"'"; // cODE IS SIMILAR TO FORM1 CODE

    cmd = new SqlCommand(syntax, con);
    dr = cmd.ExecuteReader();
    dr.Read();

    BOOK1LABELFTRNCTB.Text = Book1 = dr[0].ToString();
    con.Close();

    // To get Book 2
    con.Open();

    syntax = "SELECT BOOK2 FROM BORROWER WHERE BRID='" +
BRIDTEXTBTRNCTB.Text + "'"; // cODE IS SIMILAR TO FORM1 CODE

    cmd = new SqlCommand(syntax, con);
    dr = cmd.ExecuteReader();
    dr.Read();

    BOOK2LABELFTRNCTB.Text = Book2 = dr[0].ToString();
    con.Close();
}

```

```

    }
}
}

```

DEPARTMENT USER CONTROL CODE :-

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Data.SqlClient;

namespace Library_Management_System
{
    public partial class DEPARTMENT_USER_CONTROL : UserControl
    {
        private static DEPARTMENT_USER_CONTROL _instance;

        public static DEPARTMENT_USER_CONTROL Instance
        {
            get
            {
                if (_instance == null)
                {
                    _instance = new DEPARTMENT_USER_CONTROL();
                }
                return _instance;
            }
        }

        public DEPARTMENT_USER_CONTROL()
        {
            InitializeComponent();

            SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=\\|DataDirectory|\\DBMS1.mdf\\";Integrated
Security=True");
            SqlCommand cmd;

```



```

        public void refresh_DataGridView()           // SELECTING ALL THE DATA AND DISPLAYING IT
ON THE SCREEN
        {
            try
            {
                SqlCommand cmd = new SqlCommand("SHOW_ALL_DEPT_DATA_SP", con);           //
The STORED PROCEDURE- SHOW ALL BORROWER'S DATA
                                                    //
IS YET TO BE CODED
                cmd.CommandType = CommandType.StoredProcedure;

                SqlDataAdapter DA = new SqlDataAdapter(cmd);
                DataSet DS = new DataSet();
                DA.Fill(DS);
                con.Open();
                try
                {
                    cmd.ExecuteNonQuery();
                }
                catch (Exception exp)
                {
                    MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" + exp);
                }
                con.Close();

                DEPTTABLEDEPTTB.DataSource = DS.Tables[0];           // Below is code for
front end view of the books menu
                this.DEPTTABLEDEPTTB.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
                this.DEPTTABLEDEPTTB.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
                this.DEPTTABLEDEPTTB.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

            }
            catch (Exception exp)
            {
                MessageBox.Show(" " + exp);
            }
        }

private void REFRESHBTNDEPTTB_Click(object sender, EventArgs e)
{
    refresh_DataGridView();
}

private void CLEARBTNDEPTTB_Click(object sender, EventArgs e)
{
    DEPIDTEXTBDEPTTB.Text = " ";
    DEPTIDTEXTB2DEPTTB.Text = " ";
    NAMETEXTBDEPTTB.Text = " ";
    RACKTEXTBDEPTTB.Text = " ";
}

private void SEARCHDEPTBTNTB_Click(object sender, EventArgs e)
{

```

```

try                                     // SEARCHING FOR A SPECIFIC TUPLE OR RECORD IN THE TABLE
{
    SqlCommand cmd = new SqlCommand("SEARCH_DEPT_SP", con);
    cmd.CommandType = CommandType.StoredProcedure;
    cmd.Parameters.AddWithValue("@DEP_ID", DEPIDTEXTBDEPTTB.Text);

    SqlDataAdapter DA = new SqlDataAdapter(cmd);
    DataSet DS = new DataSet();
    DA.Fill(DS);

    con.Open();
    try
    {
        cmd.ExecuteNonQuery();
    }
    catch (Exception exp)
    {
        MessageBox.Show("<<<INVALID SQL OPERATION>>>: \n" + exp);
    }
    con.Close();

    DEPTTABLEDEPTTB.DataSource = DS.Tables[0];
    this.DEPTTABLEDEPTTB.Columns[0].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
    this.DEPTTABLEDEPTTB.Columns[1].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;
    this.DEPTTABLEDEPTTB.Columns[2].AutoSizeMode =
DataGridViewAutoSizeColumnMode.DisplayedCells;

}
catch (Exception exp)
{
    MessageBox.Show(" " + exp);
}
}

private void UPDATEBTNDEPTTB_Click(object sender, EventArgs e)
{
    SqlCommand cmd = new SqlCommand(" ", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@DEP_ID", DEPTIDTEXTB2DEPTTB.Text);
    cmd.Parameters.AddWithValue("@NAME", NAMETEXTBDEPTTB.Text);
    cmd.Parameters.AddWithValue("@RACK_CODE", RACKTEXTBDEPTTB.Text);

    con.Open();
    try
    {
        cmd.ExecuteNonQuery();
    }
    catch (Exception exp)                                     // To handle exceptions of SQL
    {
        MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
    }
}

```

QUERIES

```

    }
    con.Close();

    refresh_DataGridView();
}

private void ADDDEPTBNTB_Click(object sender, EventArgs e)
{
    SqlCommand cmd = new SqlCommand("ADD_DEPT_SP", con);
    cmd.CommandType = CommandType.StoredProcedure;

    cmd.Parameters.AddWithValue("@DEP_ID", DEPTIDTEXTB2DEPTTB.Text);
    cmd.Parameters.AddWithValue("@NAME", NAMETEXTBDEPTTB.Text);
    cmd.Parameters.AddWithValue("@RACK_CODE", RACKTEXTBDEPTTB.Text);

    con.Open();
    try
    {
        cmd.ExecuteNonQuery();
    }
    catch (Exception exp) // To handle exceptions of SQL
    {
        MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
    }
    con.Close();

    refresh_DataGridView();
}

private void DELETEBTNDEPTTB_Click(object sender, EventArgs e)
{
    try
    {
        SqlCommand cmd = new SqlCommand("DEPT_DELETE_SP", con);
        cmd.CommandType = CommandType.StoredProcedure;

        cmd.Parameters.AddWithValue("@DEP_ID", DEPTIDTEXTB2DEPTTB.Text);

        con.Open();
        try
        {
            cmd.ExecuteNonQuery();
        }
        catch (Exception exp)
        {
            MessageBox.Show("<<<INVALID SQL OPERATIONS>>>: \n" + exp);
        }
        con.Close();

        refresh_DataGridView();
    }
}

```

QUERIES

// To handle exceptions of SQL QUERIES

```

        catch (Exception exp)                // To handle exceptions of C SHARP
        {
            MessageBox.Show(" " + exp);
        }
    }

    private void DEPARTMENT_USER_CONTROL_Load(object sender, EventArgs e)
    {
        refresh_DataGridView();
    }
}

```

ABOUT USER CONTROL CODE :-

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Drawing;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Library_Management_System
{
    public partial class ABOUT_USERCONTROLS : UserControl
    {
        private static ABOUT_USERCONTROLS _Instance;

        public static ABOUT_USERCONTROLS Instance
        {
            get
            {
                if (_Instance == null)
                {
                    _Instance = new ABOUT_USERCONTROLS();
                }
                return _Instance;
            }
        }
        public ABOUT_USERCONTROLS()
        {
            InitializeComponent();
        }

        private void ABOUT_USERCONTROLS_Load(object sender, EventArgs e)
        {
        }
    }
}

```

SOFTWARE SNAPSHOTS

LOGIN FORM :-

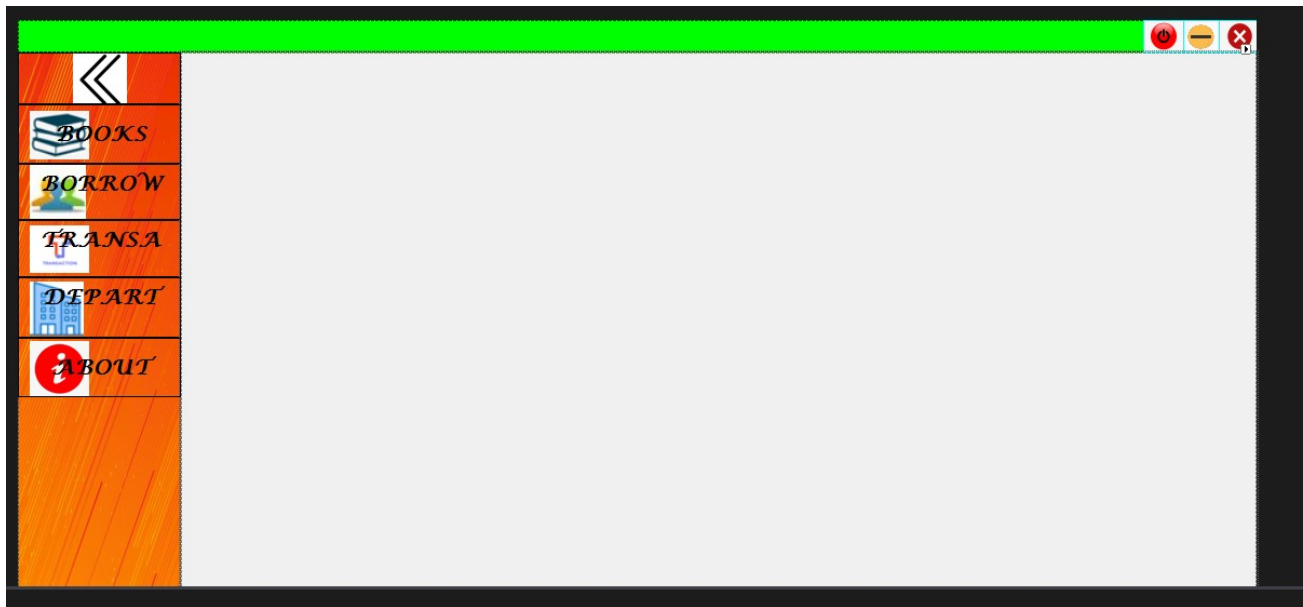


The screenshot shows a web application window titled "LIBRARY MANAGEMENT SYSTEM". The background is a blurred image of a library with bookshelves and a stack of books on a wooden table. The login form consists of the following elements:

- LIBRARY MANAGEMENT SYSTEM**: Title of the application, displayed in blue italicized text.
- USERNAME**: Label for the username field, displayed in red italicized text.
- : Input field for the username.
- PASSWORD**: Label for the password field, displayed in red italicized text.
- : Input field for the password.
- LOG IN**: A blue button with white italicized text.
- Credentials entered are wrong**: A yellow error message displayed below the login button.

At the bottom of the window, there are tabs for "Error List ..." and "Output".

APPBODY FORM :-



BOOKS USER CONTROL :-

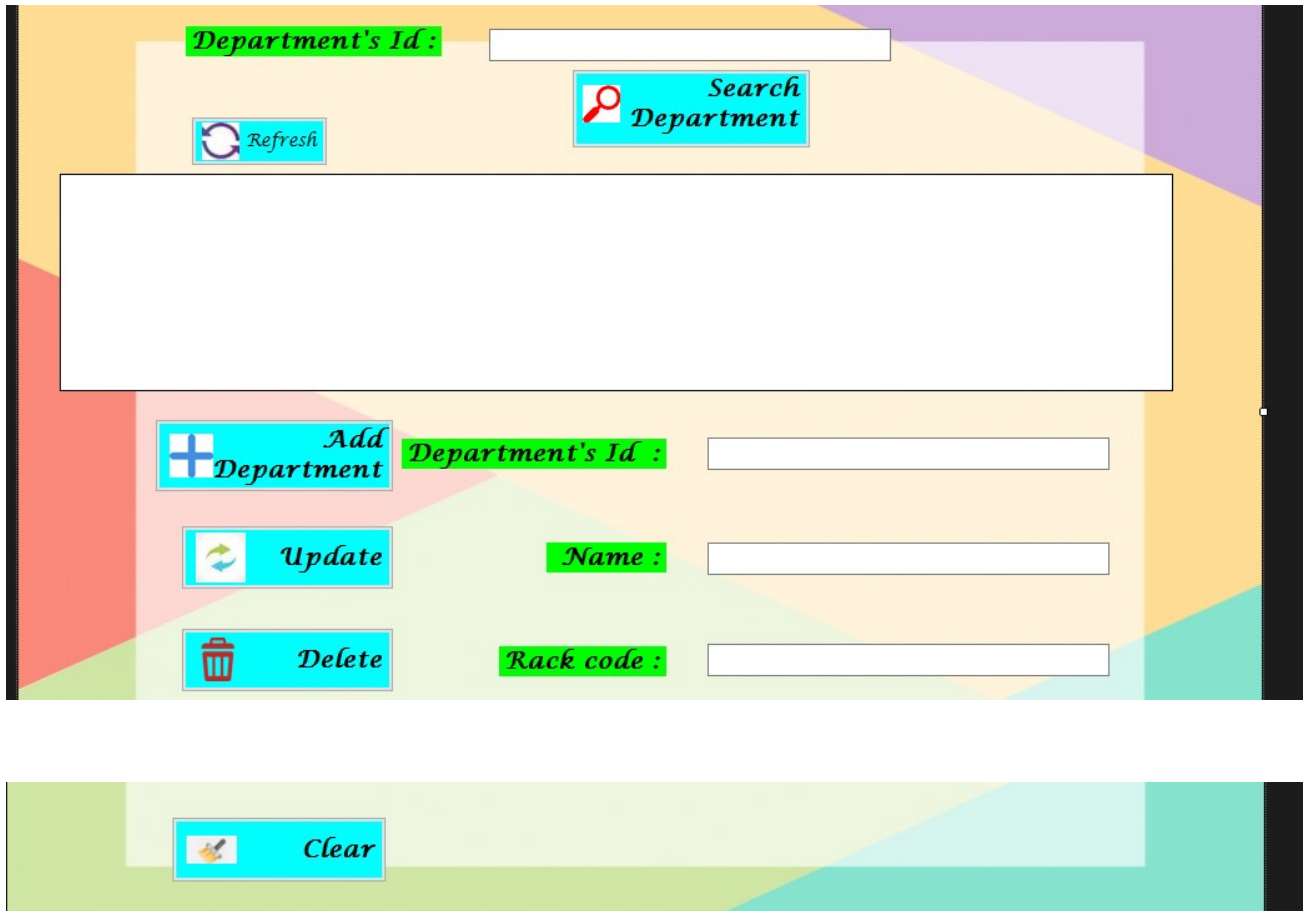
 A screenshot of a web form titled 'BOOKS USER CONTROL'. The form has a colorful background with a geometric pattern. At the top, there is a label 'Accession Number :' followed by a text input field. Below this are two buttons: 'Refresh' (with a circular arrow icon) and 'Search' (with a magnifying glass icon). In the center is a large white rectangular area. Below this area are four buttons: 'Add New' (with a plus icon), 'Update' (with a circular arrow icon), 'Delete' (with a trash can icon), and 'Clear' (with a hand icon). At the bottom, there are six input fields arranged in three rows and two columns. The labels for these fields are 'Acc No :', 'Name :', 'Author :', 'Isbn :', 'Publisher :', and 'Department :'. Each label is in a yellow box, and each input field is an orange rectangle.

BORROWERS USER CONTROL :-

Borrower's Id :	
 Refresh	 Search Borrowers
 Add Borrower	Borrower's Id :
 Update	Name :
 Delete	Address :
 Clear	Phone :

PTO

DEPARTMENT USER CONTROL CODE :-



The form is titled "DEPARTMENT USER CONTROL CODE :-". It features a header section with a "Department's Id" label and a text input field. Below this is a "Refresh" button with a circular arrow icon and a "Search Department" button with a magnifying glass icon. A large, empty rectangular box is positioned below the search button. The main section contains three rows of controls. The first row has an "Add Department" button with a plus icon, followed by a "Department's Id" label and a text input field. The second row has an "Update" button with a circular arrow icon, followed by a "Name" label and a text input field. The third row has a "Delete" button with a trash can icon, followed by a "Rack code" label and a text input field. A "Clear" button with an eraser icon is located at the bottom left of the form.

Department's Id :

 Refresh

 Search Department

 Add Department

Department's Id :

 Update

Name :

 Delete

Rack code :

 Clear

PTO

ABOUT USER CONTROL :-

